ASSESSMENT TOOL 1

ITA0609 - Machine Learning

Sameer Ahmed G

192421154

1. Combining and Validating Large Pandas DataFrames

Introduction

In a real-world data analysis system, customer information is often distributed across multiple datasets. For example, one DataFrame may contain customer transaction information while another contains demographic information. These datasets must be combined carefully so that valid customer records are preserved and duplicate customer IDs are not introduced. Pandas provides powerful operations such as merge(), concat(), join(), filtering, grouping, sorting, and indexing for performing these tasks. In this case, the customer ID acts as the common key between the transaction and demographic DataFrames. The objective is to combine both datasets while maintaining one valid customer record per customer, preserving required records, and validating the final DataFrame.

Example DataFrames

Consider two DataFrames. The first DataFrame, transactions, contains transaction-related information.

Customer_ID	Transaction_ID	Amount
C101	T001	2500
C102	T002	1800
C103	T003	3200
C104	T004	1500

The second DataFrame, demographics, contains customer information.

Customer_ID	Age	City	Gender
C101	25	Chennai	M
C102	31	Bangalore	F
C103	28	Mumbai	M
C104	42	Delhi	F

The common column between the two DataFrames is Customer_ID.

Step 1: Load the Data

The datasets can be loaded into Pandas using read_csv().

```
import pandas as pd


transactions = pd.read_csv("transactions.csv")

demographics = pd.read_csv("demographics.csv")
```

Before combining the DataFrames, their structure should be inspected.

```
print(transactions.head())

print(demographics.head())


print(transactions.info())

print(demographics.info())
```

The info() function helps identify incorrect data types, missing values, and unexpected columns.

Step 2: Check Customer ID Uniqueness

The first constraint is that there must be no unwanted duplication of customer IDs. Duplicate IDs can be identified using duplicated().

```
duplicate_transactions = transactions[

    transactions["Customer_ID"].duplicated(keep=False)

]


duplicate_demographics = demographics[

    demographics["Customer_ID"].duplicated(keep=False)

]
```

If customer IDs are expected to occur only once in the demographic table, duplicates should be removed or investigated.

```
demographics = demographics.drop_duplicates(

    subset=["Customer_ID"],

    keep="first"

)
```

However, transaction data may legitimately contain multiple transactions for one customer. Therefore, it is important to distinguish between **duplicate customer records** and **multiple valid transactions**. If the requirement is one row per customer, transaction records should first be aggregated by customer ID rather than simply deleting rows.

Step 3: Aggregate Transaction Data

If several transactions belong to the same customer, they can be grouped using groupby().

```
customer_transactions = (
    transactions
    .groupby("Customer_ID", as_index=False)
    .agg(
        Total_Amount=("Amount", "sum"),
        Transaction_Count=("Transaction_ID", "count")
    )
)
```

This produces one summarized transaction record for each customer.

Customer_ID	Total_Amount	Transaction_Count
C101	2500	1
C102	1800	1
C103	3200	1
C104	1500	1

Step 4: Merge the DataFrames

The two DataFrames can now be combined using merge().

```
combined = pd.merge(
    customer_transactions,
    demographics,
    on="Customer_ID",
    how="outer",
    validate="one_to_one"
)
```

An **outer merge** is used when the requirement is to preserve all valid records from both datasets. If only customers appearing in both datasets are required, an inner merge can be used.

The validate="one_to_one" argument is especially useful because Pandas will raise an error if the merge unexpectedly produces a one-to-many or many-to-many relationship.

Step 5: Identify Missing Entries

After merging, missing values can be identified using isna().

```
missing_records = combined[
    combined.isna().any(axis=1)
]
```

The source of missing entries can be identified using an indicator column.

```
check = pd.merge(
    customer_transactions,
    demographics,
    on="Customer_ID",
    how="outer",
    indicator=True
)


print(check["_merge"].value_counts())
```

The _merge column indicates whether each customer exists in both datasets, only in the transaction dataset, or only in the demographic dataset.

Possible results include:

```
both          950
left_only      30
right_only     20
```

This information helps identify mismatched customer IDs.

Step 6: Debug Mismatched IDs

Mismatches can occur because of spaces, different capitalization, inconsistent formatting, or incorrect identifiers. For example, C101 and c101 may represent the same customer.

The IDs should therefore be standardized before merging.

```
transactions["Customer_ID"] = (
    transactions["Customer_ID"]
```

```
    .astype(str)
    .str.strip()
    .str.upper()
)

demographics["Customer_ID"] = (
    demographics["Customer_ID"]
    .astype(str)
    .str.strip()
    .str.upper()
)
```

This removes leading and trailing spaces and ensures consistent capitalization.

Step 7: Validate the Number of Records

The number of unique customers before and after merging should be checked.

```
print(
    customer_transactions["Customer_ID"].nunique()
)

print(
    demographics["Customer_ID"].nunique()
)

print(
    combined["Customer_ID"].nunique()
)
```

The number of unique IDs in the final result should be consistent with the selected merge strategy. Unexpected increases usually indicate a many-to-many merge caused by duplicate keys.

Step 8: Validate Using Grouping

Grouping can be used to detect whether any customer appears more than once in the final DataFrame.

```
id_counts = (
    combined
    .groupby("Customer_ID")
    .size()
    .reset_index(name="Count")
)


duplicates = id_counts[
    id_counts["Count"] > 1
]
```

If the final DataFrame is expected to contain exactly one row per customer, the duplicates DataFrame should be empty.

Step 9: Filtering Valid Records

Invalid records can be identified using filtering conditions. For example, transaction amounts should not normally be negative.

```
invalid_amounts = combined[
    combined["Total_Amount"] < 0
]
```

Similarly, impossible ages can be identified.

```
invalid_age = combined[
    (combined["Age"] < 0) |
    (combined["Age"] > 120)
]
```

Such records should be investigated and corrected or removed according to the data quality requirements.

Step 10: Sorting and Indexing

After validation, the DataFrame can be sorted by customer ID.

```
combined = combined.sort_values(
    by="Customer_ID"
```

)

The index should then be reset so that the final DataFrame has a consistent sequential index.

```
combined = combined.reset_index(drop=True)
```

The final structure can be checked using:

```
print(combined.head())
print(combined.index)
```

Correctness Validation

The final DataFrame should satisfy the following conditions:

1. Customer IDs are correctly standardized.
2. No unwanted duplicate customer IDs exist.
3. All required valid records are preserved.
4. Missing values are identified and handled.
5. Transaction records are correctly aggregated.
6. The final DataFrame has a consistent index.
7. Sorting is performed using the customer ID.
8. Data types are correct.
9. Invalid values are detected through filtering.
10. The merge relationship is validated.

Debugging Mismatched or Missing Entries

If the number of records after merging is unexpectedly large, duplicate keys should be investigated using duplicated() and groupby(). If records are missing, an outer merge with indicator=True should be used to determine which dataset contains the unmatched IDs. Whitespace and capitalization should be standardized before merging. Data types should also be checked because one DataFrame may store customer IDs as strings while another may contain numeric values. Missing demographic values should not automatically be replaced without investigating their source. Maintaining a separate validation report containing unmatched IDs and missing fields makes the debugging process easier.

Conclusion

Combining large Pandas DataFrames requires more than simply joining two tables. The customer ID must be standardized and validated before the merge, and transaction data should be aggregated when the final requirement is one record per customer. An outer merge with validation and merge indicators helps preserve valid records and identify mismatched entries. Filtering and grouping can detect invalid values and duplicate IDs, while sorting and reset_index() ensure that the final DataFrame has a consistent structure. Therefore, Pandas provides an efficient and reliable framework for combining large customer datasets while maintaining data integrity and making mismatches easier to debug.

2. Complete Data Analysis Pipeline Using Pandas and Visualization

Introduction

A complete data analysis pipeline consists of several stages, beginning with data acquisition and ending with visualization and interpretation. In a real-world application, data may be distributed across multiple CSV, Excel, or other files, and these files may contain inconsistent column names, missing values, incorrect data types, duplicate records, or corrupted entries. Pandas provides efficient DataFrame-based operations for loading, cleaning, transforming, filtering, grouping, aggregating, and sorting data. Visualization libraries such as Matplotlib and Seaborn can then be used to convert processed information into meaningful graphical representations. The objective is to design an efficient pipeline that handles multiple files, performs reliable data processing, supports user-defined filtering, produces meaningful plots, and includes debugging mechanisms for corrupted or inconsistent input files.

Pipeline Architecture

Multiple Data Files

↓

File Validation

↓

Efficient Data Loading

↓

Data Inspection

↓

Data Cleaning

↓

Data Transformation

↓

User-Defined Filtering

↓

Grouping and Aggregation

↓

Sorting

↓

Visualization

↓

Interpretation and Reporting

Step 1: Identify and Read Multiple Files

Python's glob module can be used to identify all files matching a particular pattern.

```
from pathlib import Path
import pandas as pd

files = list(Path("data").glob("*.csv"))
```

The files can then be loaded into Pandas.

```
dataframes = []

for file in files:
    df = pd.read_csv(file)
    dataframes.append(df)
```

The individual DataFrames can be combined using pd.concat().

```
data = pd.concat(
    dataframes,
    ignore_index=True
)
```

The ignore_index=True option creates a new continuous index after combining the files.

Step 2: Efficient File I/O

For large datasets, loading the entire file into memory may not be efficient. Pandas supports chunk-based reading using the chunksize parameter.

```
for chunk in pd.read_csv(
    "large_file.csv",
    chunksize=50000
):
    process(chunk)
```

Only the required columns should be loaded when possible.

```
df = pd.read_csv(
    "sales.csv",
    usecols=[
        "Date",
        "Product",
        "Region",
        "Sales"
    ]
)
```

Specifying data types can also reduce memory consumption.

```
df = pd.read_csv(
    "sales.csv",
    dtype={
        "Product": "string",
        "Region": "string",
        "Sales": "float32"
    }
)
```

These techniques improve file-reading performance and reduce memory usage.

Step 3: Inspect the Data

Before processing the data, the structure should be examined.

```
print(data.head())
print(data.shape)
```

```
print(data.info())

print(data.describe())
```

The column names should also be checked.

```
print(data.columns)
```

This step helps identify unexpected columns, incorrect data types, missing fields, and formatting problems.

Step 4: Standardize Column Names

Different input files may use different column naming conventions. Column names should therefore be standardized.

```
data.columns = (
    data.columns
    .str.strip()
    .str.lower()
    .str.replace(" ", "_")
)
```

For example, Sales Amount, sales amount, and SALES_AMOUNT can all be converted to sales_amount.

Step 5: Handle Missing Values

Missing values can be identified using:

```
print(data.isnull().sum())
```

Depending on the nature of the variable, missing values can be removed or imputed.

```
data["sales"] = data["sales"].fillna(
    data["sales"].median()
)
```

Categorical values can be filled using the mode or a suitable category such as "Unknown".

```
data["region"] = data["region"].fillna("Unknown")
```

Step 6: Remove Duplicate Records

Duplicate rows can distort aggregation and visualization results.

```
data = data.drop_duplicates()
```

If duplicates are determined using specific columns, those columns can be specified.

```
data = data.drop_duplicates(
    subset=["date", "product", "region"]
)
```

Step 7: Correct Data Types

Dates should be converted to Pandas datetime format.

```
data["date"] = pd.to_datetime(
    data["date"],
    errors="coerce"
)
```

Numerical columns should be converted using:

```
data["sales"] = pd.to_numeric(
    data["sales"],
    errors="coerce"
)
```

The errors="coerce" option converts invalid values into missing values, which can then be identified and handled.

Step 8: User-Defined Filtering

The pipeline should allow users to specify conditions for selecting relevant records. For example, a user may request all transactions with sales greater than ₹10,000.

```
filtered = data[
    data["sales"] > 10000
]
```

Multiple conditions can be combined.

```
filtered = data[
    (data["sales"] > 10000) &
    (data["region"] == "South")
]
```

The system can also accept variables from the user.

```
minimum_sales = 10000
selected_region = "South"
```

```
filtered = data[
    (data["sales"] >= minimum_sales) &
    (data["region"] == selected_region)
]
```

Step 9: Grouping and Aggregation

Pandas groupby() is used to summarize the processed data.

```
regional_sales = (
    filtered
    .groupby("region")["sales"]
    .sum()
    .reset_index()
)
```

Multiple aggregation functions can also be applied.

```
summary = (
    data
    .groupby("region")
    .agg(
        Total_Sales=("sales", "sum"),
        Average_Sales=("sales", "mean"),
        Transactions=("sales", "count")
    )
    .reset_index()
)
```

This produces a compact summary of sales performance by region.

Step 10: Sorting

The aggregated results can be sorted to identify the highest-performing regions.

```
summary = summary.sort_values(
    by="Total_Sales",
```

```
    ascending=False
)
```

Sorting after aggregation makes the analysis easier to interpret and allows the visualization to display the most important categories first.

Step 11: Visualization

Visualization converts processed data into graphs that make trends and relationships easier to understand. Matplotlib can be used for creating charts.

```
import matplotlib.pyplot as plt


plt.bar(
    summary["region"],
    summary["Total_Sales"]
)


plt.xlabel("Region")
plt.ylabel("Total Sales")
plt.title("Sales by Region")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

A line graph can be used for time-series analysis.

```
monthly_sales = (
    data
    .groupby("date")["sales"]
    .sum()
)


monthly_sales.plot(
    kind="line",
    figsize=(10, 5)
```

```
)

plt.xlabel("Date")
plt.ylabel("Sales")
plt.title("Sales Trend Over Time")
plt.tight_layout()
plt.show()
```

A histogram can be used to understand the distribution of transaction amounts.

```
plt.hist(
    data["sales"].dropna(),
    bins=20
)

plt.xlabel("Sales")
plt.ylabel("Frequency")
plt.title("Distribution of Sales")
plt.show()
```

Meaningful Visualization Selection

The choice of graph should depend on the analytical objective. Bar charts are suitable for comparing categories, line charts are appropriate for time-series trends, histograms show distributions, scatter plots show relationships between numerical variables, and box plots help identify variation and outliers. The visualization should contain meaningful axis labels, an informative title, appropriate units, and readable category labels.

Step 12: Interpretation

Visualization should not be treated as the final step without interpretation. For example, if the regional sales graph shows that the South region has the highest total sales, the analyst can investigate the reasons for its performance. Similarly, if the time-series graph shows a sudden decrease in sales, additional investigation can determine whether the decrease is related to seasonality, supply problems, pricing changes, or external events.

Debugging Corrupted Files

Input files may be corrupted or contain malformed rows. Pandas provides error-handling options while reading CSV files.

```
try:
    df = pd.read_csv(file)
except Exception as e:
    print(f"Error reading {file}: {e}")
```

Files that cannot be read should be logged and skipped rather than terminating the entire pipeline.

```
valid_data = []


for file in files:
    try:
        df = pd.read_csv(file)
        valid_data.append(df)
    except Exception as e:
        print(
            f"Skipping {file}: {e}"
        )
```

This allows the remaining valid files to be processed.

Handling Inconsistent Formats

Different files may use different date formats, column names, or numerical representations. Dates should be standardized using pd.to_datetime(), numerical values should be converted using pd.to_numeric(), and column names should be normalized before concatenation. For example, values such as "₹10,500" should be cleaned before numerical analysis.

```
data["sales"] = (
    data["sales"]
    .astype(str)
    .str.replace("₹", "", regex=False)
    .str.replace(",", "", regex=False)
)
```

```
data["sales"] = pd.to_numeric(
    data["sales"],
    errors="coerce"
)
```

Validation and Debugging

The final dataset should undergo validation checks before analysis.

```
assert data["sales"].notna().all()
assert data["region"].notna().all()
```

Duplicate records can be checked using duplicated(), missing values using isnull(), invalid values using filtering, and unexpected categories using unique().

```
print(data["region"].unique())
print(data.isnull().sum())
print(data.duplicated().sum())
```

These checks ensure that incorrect data does not silently affect the final results.

Optimization Techniques

The pipeline can be optimized by reading only required columns, specifying appropriate data types, processing large files in chunks, avoiding unnecessary copies of DataFrames, using vectorized Pandas operations instead of Python loops, filtering data before expensive aggregation operations, and grouping only the required columns. These techniques reduce memory consumption and improve execution speed.

Complete Pipeline Example

```
from pathlib import Path
import pandas as pd
import matplotlib.pyplot as plt


files = Path("data").glob("*.csv")
frames = []
```

```
for file in files:
    try:
        df = pd.read_csv(file)

        df.columns = (
            df.columns
            .str.strip()
            .str.lower()
            .str.replace(" ", "_")
        )

        frames.append(df)

    except Exception as e:
        print(f"Error reading {file}: {e}")

data = pd.concat(
    frames,
    ignore_index=True
)

data = data.drop_duplicates()

data["date"] = pd.to_datetime(
    data["date"],
    errors="coerce"
)
```

```
data["sales"] = pd.to_numeric(
    data["sales"],
    errors="coerce"
)

data = data.dropna(
    subset=["date", "sales"]
)

minimum_sales = 10000

filtered = data[
    data["sales"] >= minimum_sales
]

summary = (
    filtered
    .groupby("region")
    .agg(
        Total_Sales=("sales", "sum"),
        Average_Sales=("sales", "mean")
    )
    .reset_index()
    .sort_values(
        "Total_Sales",
        ascending=False
    )
)
```

```
plt.bar(
    summary["region"],
    summary["Total_Sales"]
)

plt.xlabel("Region")
plt.ylabel("Total Sales")
plt.title("Regional Sales Analysis")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

Conclusion

A complete Pandas-based data analysis pipeline should systematically perform file loading, validation, preprocessing, transformation, filtering, aggregation, sorting, visualization, and interpretation. Efficient file I/O can be achieved using selective column loading, appropriate data types, and chunk processing for large files. Pandas operations such as groupby(), sort_values(), filtering, and vectorized transformations provide efficient data processing. Visualization using appropriate charts helps identify trends, comparisons, distributions, and relationships in the processed data. Corrupted files and inconsistent formats can be handled using exception handling, data type conversion, validation checks, and logging. Therefore, a properly designed Pandas pipeline provides an efficient, reliable, and reproducible framework for converting multiple raw data files into meaningful analytical insights.